Without parentheses or arguments, the function itself is being evaluated, instead of the result of invoking the function:

```
sayHello
# [Function]
```

Evaluates to a function.

In general, you should omit parentheses where possible.

Implicit return

The return value for a function is the last expression evaluated when the function is invoked. In JavaScript a function requires an explicit `return` statement indicating the return value. If there is no such statement, then the return value is `undefined`:

```
returnsUndefined = function(a,b) {
  a + b;
};

returnsUndefined();
// undefined
```

In JavaScript the result of invoking a function that has no return statement is undefined.

In CoffeeScript the only way to return `undefined` from a function is on purpose, by having it as the last expression evaluated:

```
returnsUndefined = ->
  return undefined

returnsUndefined()
# undefined
```

In CoffeeScript a function returns undefined only if you make it.

Return from inside expression

Remember the `countWords` function?

```
countWords = (text, delimiter) ->
  words = text.split delimiter
  words.length
```

The last expression evaluated is `words.length`. Suppose that this function doesn't quite do what you need it to—it has several problems. The